PORTFOLIO OVERVIEW

Haris Khan

Full-Stack Software Engineer

Full-stack software engineer specializing in AI-powered web applications, e-commerce platforms, and enterprise management systems. Across React, Node.js, Python ML/AI, and modern frameworks, delivering complete solutions from concept to deployment, turning business challenges into scalable, user-friendly applications.

SERVICES

- Full-Stack Web Development
- E-commerce Solutions
- AI/ML Integration
- Desktop Applications
- Mobile Development
- Technical Consulting

INDUSTRIES SERVED

- E-commerce & Retail
- SaaS Platforms
- Robotics & AI
- Automotive Services
- Finance & Inventory Management
- Food & Hospitality

IN THIS PORTFOLIO

13 project case studies: 11 freelance projects and 2 hobby projects. One project per page, spanning desktop operations, commerce, AI workflows, websites, and simulations.

FEATURED PROJECTS

FREELANCE PROJECTS

01 KontrolAlt
02 EchoNote
03 Upwork Cowork Agent
04 SAFISH PLUS POS
05 SAFISH PLUS DISTRIBUTOR ERP
06 3LT Journal Theme
07 Pulse Robot
08 Singhar By Sidra
09 Goodman Group
10 Inventory & Finance Management System
11 Trulle Portal

HOBBY PROJECTS

12 Trend Finder
13 Software Server Queue Simulation Dashboard

KontrolAlt

A production creator-discovery and growth-intelligence platform finding, scoring, and monitoring emerging Rumble and Substack channels.

INDUSTRY / SYSTEM	SCOPE	STATUS
Creator Discovery / Growth Intelligence	**Frontend · API · Scrapers · Infrastructure**	**Deployed**

THE CHALLENGE

Creator partnership teams spend time searching platforms manually, guessing which channels are growing, and losing track of outreach. Hand scouting provides inconsistent engagement records and scales poorly, especially on platforms that resist automation. Teams need continuous discovery and shared, inspectable prospect intelligence.

THE SOLUTION

Built independently deployed discovery/scraping, analytics, client dashboard, and operations services connected through HTTP contracts, Celery job queues, and shared Supabase/PostgreSQL data. Next.js serves the interface, FastAPI handles business operations and authentication, and separate browser workers perform scraping. Snapshot history supports engagement tiers, growth velocity, AI narratives, and similar-channel matching.

HOW IT WORKS

1. Keyword/seed search, bulk URL import, and fuzzy creator-name resolution feed discovery and prospect intake.
2. Workers use stealth-configured Playwright sessions, channel-pinned residential proxies, proxy health/quarantine, and Cloudflare challenge detection with retries.
3. Staggered task chains support scheduled/on-demand scraping; logged attempts and worker health expose operational failures and data gaps.

KEY HIGHLIGHTS

- ✓ Rumble and Substack discovery with an architecture designed for additional scraper modules and queues.
- ✓ CSV/Excel exports turn filtered views into outreach and reporting shortlists.
- ✓ 30-day and 90-day view/comment trend comparisons identify growth velocity.
- ✓ Shared partnership states track current partners, previous hires, and do-not-contact channels.
- ✓ Combinable platform, topic, engagement-tier, subscriber, view/comment, and last-active filters.
- ✓ AI channel reports describe content focus, audience profile, and partnership fit.
- ✓ Lookalike matching expands a shortlist from topic tags and engagement profiles.
- ✓ Authenticated roles, admin controls, manual job triggers, live worker monitoring, and administrative audit logs.

TECHNOLOGY STACK

Frontend	Next.js 16 App Router, TypeScript, Tailwind CSS
API	FastAPI / Python, Supabase Auth / JWT
Workers	Celery, Redis, scheduled and on-demand task chains
Scraping	Python Playwright, residential proxy pool, challenge/retry handling
Data	Supabase / PostgreSQL shared by API and scraper workers
Infrastructure	Vercel, Render / Docker, Upstash

VIDEO WALKTHROUGH

Watch project video

EchoNote

A full-stack meeting-intelligence application turning recorded conversations into speaker-attributed transcripts, structured summaries, and connected follow-up work.

INDUSTRY / SYSTEM	SCOPE	STATUS
SaaS / Meeting Intelligence	**Audio · AI · Tasks · Collaboration**	**Deployed**

THE CHALLENGE

Meeting decisions and commitments become scattered across recordings, notes, and task trackers. Replaying audio is slow, manual notes divide attention, and summaries often lose speaker context or follow-up ownership. Teams need a connected record of the discussion and the work that follows.

THE SOLUTION

Built a React and Express application spanning browser recording/upload, asynchronous audio analysis, searchable meetings, linked tasks, and shared workspaces. An in-process FIFO worker coordinates Python audio preparation, Deepgram transcription, local spaCy enrichment, and Groq-backed structured summaries. PostgreSQL/Prisma stores meeting context and tasks, while ownership and workspace roles protect access.

HOW IT WORKS

1. Authenticated admission validates audio type, size, and duration, creates a user-owned meeting, and returns its ID before processing completes.
2. Python prepares mono 16 kHz PCM WAV; Deepgram produces punctuated text and timestamped speaker segments; spaCy extracts language signals.
3. Groq proposes summaries, decisions, actions, and next steps. Persisted stages and polling expose progress, with backoff retries and degraded NLP fallback.

KEY HIGHLIGHTS

- ✓ Browser microphone recording, supported audio uploads, metadata, categories, and searchable meeting status.
- ✓ Executive summaries, decisions, topics, suggested next steps, and meeting-linked action candidates.
- ✓ Summary sharing and owner/editor/viewer team workspace membership.
- ✓ Google Calendar events provide scheduling and meeting metadata context.
- ✓ Speaker-attributed transcripts with timestamps, confidence context, and editable speaker mappings.
- ✓ Tasks retain ownership, deadlines, priorities, and TODO / IN_PROGRESS / DONE status.
- ✓ Liveblocks collaborative editing authorized against meeting ownership or workspace role.
- ✓ Optional Jira synchronization and completion updates through email, Web Push, or configured Slack webhook.

TECHNOLOGY STACK

Frontend	React 18, React Router, HeroUI, Tailwind CSS, RecordRTC
API / identity	Node.js, Express, JWT, Google OAuth
Processing	In-process FIFO worker, Python audio tools, spaCy
AI services	Deepgram speaker transcription, Groq-backed language-model service
Data	PostgreSQL, Prisma, Supabase or local audio files
Collaboration	Liveblocks, Google Calendar, optional Jira / Slack / email / Web Push

VIDEO WALKTHROUGH

Watch project video

Upwork Cowork Agent

An AI-assisted operations dashboard connecting explainable job discovery, portfolio-grounded proposals, and durable human review.

INDUSTRY / SYSTEM	SCOPE	STATUS
Freelance Operations / Internal Tool	**Scout · Writer · Critic · Proposal History**	**Implemented**

THE CHALLENGE

Selecting freelance work requires assessing skills, budgets, client history, competition, and eligibility. Disconnected searches, AI drafts, and review notes make decisions difficult to trace, while generating a proposal for every shortlisted job spends model budget before an operator chooses what to pursue.

THE SOLUTION

Built a workspace-scoped React and FastAPI modular monolith backed by PostgreSQL. Scout discovers and scores jobs through Upwork MCP; an operator explicitly starts Writer, optional Critic, and revisions. Portfolio evidence grounds drafts, while saved versions, structured reviews, remarks, and approvals preserve the decision trail. Provider/model assignments resolve independently for Writer and Critic without silent fallback.

HOW IT WORKS

1. Manual or enabled two-hour discovery runs apply revisioned scoring rules, competition thresholds, location policies, and provenance tracking.
2. Explicit commands enqueue generation. PostgreSQL claims, leases, heartbeats, and fencing tokens prevent stale-worker writes.
3. Critiques bind to saved versions; revisions are capped at two rounds. Approval records the decision and does not submit to Upwork.

KEY HIGHLIGHTS

- ✓ Explainable fit rubric initially weights skills, historical client spend, and client history at 50/30/20.
- ✓ Editable cover letters, screening answers, bid guidance, and matched portfolio evidence.
- ✓ Version-bound PASS/REVISE reviews, explicit revisions, operator remarks, and approval history.
- ✓ Account-bound sessions, CSRF protection, workspace filtering, and audit records.
- ✓ Quality states separate suitable, unsuitable, and uncertain jobs without triggering automatic writing.
- ✓ Independent Anthropic, OpenAI, Gemini, xAI, and Groq provider integrations for Writer/Critic.
- ✓ Backend-owned Upwork OAuth/PKCE, encrypted tokens and provider credentials, and token refresh.
- ✓ On-demand Connects cost, wallet balance, and boost guidance; live submission is disabled by default.

TECHNOLOGY STACK

Frontend	React 19, TypeScript, Vite, TanStack Query, Tailwind CSS 4
API	Python 3.12, FastAPI, Pydantic, session cookies, Argon2
Data / jobs	PostgreSQL, async SQLAlchemy, asyncpg, Alembic, database leases
Integrations	APScheduler 3.x, Python workers, official MCP SDK, OAuth/PKCE, encrypted vault
Delivery	Docker, Render / Vercel SPA configuration, pytest, Vitest

VIDEO WALKTHROUGH

Watch project video

SAFISH PLUS POS

A bilingual desktop fish-retail system connecting weight-based checkout, FIFO stock batches, supplier restocks, and operational reporting.

INDUSTRY / SYSTEM	SCOPE	STATUS
Fish Retail / Point of Sale	**Desktop App · Sales · Batch Inventory**	**Implemented**

THE CHALLENGE

Fish retailers sell by weight while purchase costs change between deliveries. Checkout needs an available-stock check and a clear invoice, while the shop must track remaining batches, supplier deliveries, and daily trading results. Separate sales, restock, and stock records make those figures difficult to inspect together.

THE SOLUTION

Built a locally stored English-Urdu desktop application connecting a kilogram-based cart, supplier-linked stock batches, numbered invoices, and reports. Checkout validates availability and commits FIFO batch deductions, item quantities, invoices, and sale lines in one SQLite transaction. Costs come from the oldest available batches, with invoice discounts reflected in the final amount and profit.

HOW IT WORKS

1. Supplier restocks increase inventory and create dated batches retaining purchase rate, received weight, and remaining quantity.
2. Fuzzy English/Roman-Urdu search finds fish; the operator adjusts kilogram quantities, selling rates, and the invoice discount.
3. Posting a sale consumes FIFO batches; report queries and receipt PDFs use the persisted transaction history.

KEY HIGHLIGHTS

- ✓ Weight-based multi-item checkout with manually entered selling rates and numbered invoices.
- ✓ Transactional FIFO costing keeps sale profit and stock deductions connected.
- ✓ Dashboard stock value, activity, revenue, transaction counts, and configurable low-stock alerts.
- ✓ Five report types export to UTF-8 CSV; bilingual invoice receipts generate as PDFs.
- ✓ Supplier records and delivery history retain fish, date, weight, cost, and remaining batch quantities.
- ✓ Cash/online labels classify payments; they do not process payments.
- ✓ Six report views: daily sales, sales history, inventory, product performance, trends, and supplier restocks.
- ✓ English-Urdu labels, RTL layouts, keyboard shortcuts, and cached language preference.

TECHNOLOGY STACK

Desktop	Electron 40, Node.js, context-isolated preload and named IPC
Frontend	React 19, JavaScript, Vite 7, Mantine 8, Recharts, Fuse.js
Data	SQLite, better-sqlite3, prepared queries, WAL, transactional checkout
Localization	i18next, react-i18next, English/Urdu and RTL
Output	Electron printToPDF, CSV worker, electron-builder / Windows NSIS

VIDEO WALKTHROUGH

Watch project video

SAFISH PLUS DISTRIBUTOR ERP

A bilingual desktop trading system connecting fish purchases, owned-stock sales, vendor consignments, commission settlements, and customer accounts.

INDUSTRY / SYSTEM	SCOPE	STATUS
Fish Wholesale & Distribution	**Trading · Stock · Accounts · Settlements**	**Implemented**

THE CHALLENGE

Fish wholesalers manage changing weights and rates, customer collections, transport and ice charges, vendor advances, and settlement deductions. Selling owned stock and selling on a vendor's behalf have different inventory and settlement effects. Treating both alike obscures stock ownership and outstanding balances.

THE SOLUTION

Built an English-Urdu desktop workspace with persistent purchases, sales, customer/vendor records, stock movements, settlement bills, and thirteen report views. Domain-specific preload methods connect React screens to validated main-process SQLite handlers. Owned-stock sales reduce inventory; eligible non-stock consignment lines feed a separate vendor-billing workflow.

HOW IT WORKS

1. Purchases post vendor, fish, weight, rate, concession, and payment; stock increases and the unpaid amount affects the vendor balance.
2. Sales calculate weight-based amounts plus fare and ice charges, less cash and receipts. Only stock-sourced lines reduce owned inventory.
3. Bill previews select a vendor's posted, unbilled consignment lines, deduct commission and charges, record payment, and link the settled lines.

KEY HIGHLIGHTS

- ✓ Weight-based purchases and sales, including kg/maund conversion at 1 maund = 40 kg.
- ✓ Customer/vendor master records, opening balances, transaction balances, and vendor advances.
- ✓ Thirteen reports cover recovery, ledgers, registers, concessions, net summaries, daily trading, and stock.
- ✓ PDF document generation, print layouts, and structured Excel output through native dialogs.
- ✓ Distinct owned-stock and vendor-consignment flows with linked settlement records.
- ✓ Date-range bill previews calculate commission, expenses, concession, cash paid, and balance.
- ✓ Search and correction paths reverse and reapply linked inventory and account effects.
- ✓ English-Urdu labels, RTL presentation, keyboard shortcuts, and light/dark themes.

TECHNOLOGY STACK

Desktop	Electron 40, Node.js, context-isolated preload and IPC
Frontend	React 19, JavaScript, Vite 7, Zustand, Mantine 8, Tailwind CSS 4
Data	SQLite, better-sqlite3, foreign keys, WAL, transactional creates
Localization	i18next, react-i18next, RTL and translation key-coverage checks
Output	Electron printToPDF, ExcelJS, electron-builder

VIDEO WALKTHROUGH

Watch project video

3LT Journal Theme

A custom editorial identity and responsive public journal theme connecting issue discovery, scholarly reading, and journal information.

INDUSTRY / SYSTEM	SCOPE	STATUS
Scholarly Publishing / English Studies	**Custom Design · Django Public Theme**	**Deployed**

THE CHALLENGE

A journal's identity spans its homepage, issues, article pages, and editorial information. Readers need coherent navigation, clear scholarly metadata, and comfortable long-form reading across screen sizes. The custom presentation also had to preserve Janeway's existing publishing and editorial workflows.

THE SOLUTION

Built the threelt Django sub-theme with shared template inheritance, reusable includes, page-specific compositions, responsive CSS, and lightweight JavaScript. Ink-blue foundations, pale-blue surfaces, ochre accents, Source Serif 4 headings, and Inter/system body text establish one scholarly identity. Janeway supplies data, named routes, forms, and publishing services, with OLH/Janeway fallback for unoverridden pages.

KEY HIGHLIGHTS

- ✓ Tailored homepage with journal introduction, current-issue articles, submission entry points, and Most Read presentation.
- ✓ Scholarly article layouts for authors, affiliations, abstract, keywords, rights, citation exports, and available files.
- ✓ Responsive content/sidebar layouts and a mobile drawer with Escape-close and focus restoration.
- ✓ Janeway form processing, validation, CSRF, and integration hooks remain intact.
- ✓ Year-grouped archives, numbered issue contents, jump navigation, and integrated search filters.
- ✓ Editorial profiles, structured About/Guidelines with in-page contents, contact, and account-entry pages.
- ✓ Disclosures, empty-search guidance, unavailable-content messages, and custom public error pages.
- ✓ Print styling, reduced-motion rules, and optional analytics gated by configuration and reader acceptance.

TECHNOLOGY STACK

Platform	Janeway, OLH base theme, Django template inheritance
Composition	threelt template overrides, shared includes, page-specific layouts
Presentation	Theme-local CSS, Source Serif 4, Inter/system fonts
Interactions	Vanilla JavaScript for navigation, disclosures, filters, and consent
Support	Python asset-copy utilities, journal configuration, local test-data seeder

VIDEO WALKTHROUGH

Watch project video

Pulse Robot

A responsive product-story website introducing the Atlas humanoid companion concept through animated visuals, scroll-led storytelling, and inquiry interfaces.

INDUSTRY / SYSTEM	SCOPE	STATUS
Robotics / Product Marketing	**Frontend · Motion · Responsive Story**	**Implemented**

THE CHALLENGE

A humanoid companion concept needed an approachable introduction that connected product vision, displayed details, artwork, and capability messaging. Visitors needed one continuous product story and clear inquiry surfaces. The web project presents the Atlas concept; it does not implement or verify robot functionality.

THE SOLUTION

Built a React frontend composed of reusable landing-page sections, local media, and browser-driven interactions. A local Lottie hero retains static robot artwork as fallback. IntersectionObserver triggers reveals, while a requestAnimationFrame-coordinated scroll listener advances a three-card sticky vision sequence. Local menu and form state provide responsive navigation and demonstrable feedback without an application backend.

KEY HIGHLIGHTS

- ✓ Branded hero, three-card vision sequence, product details, image showcase, and six capability themes.
- ✓ Local Lottie animation with artwork fallback and desktop hero pointer/scroll effects.
- ✓ Inquiry form validates name and email, accepts an optional company, shows a toast, and resets locally.
- ✓ React Router supplies the landing route and a client-side not-found page.
- ✓ Responsive desktop/mobile header, mobile-menu body-scroll locking, and section links with header offsets.
- ✓ Scroll-driven stacked cards and viewport reveal effects support progressive storytelling.
- ✓ Newsletter UI demonstrates loading and confirmation with a simulated one-second delay.
- ✓ Section components, Tailwind tokens, local assets, icons, and shared feedback primitives form a reusable base.

TECHNOLOGY STACK

Application	React 18, TypeScript, Vite 5, React Router DOM 6
Presentation	Tailwind CSS 3, PostCSS, CSS variables, local public assets
Motion	lottie-react, IntersectionObserver, browser events, requestAnimationFrame
Feedback	Sonner, Lucide, Radix-based UI primitives, React state/effects
Scope	Browser frontend; inquiry/signup demos do not send or persist submissions

VIDEO WALKTHROUGH

Watch project video

Singhar By Sidra

An editorial-first fashion storefront connecting occasion-led discovery, Shopify commerce, managed homepage content, and customer accounts.

INDUSTRY / SYSTEM	SCOPE	STATUS
Luxury Fashion / E-Commerce	**Shopify Hydrogen · Editorial CMS**	**Deployed**

THE CHALLENGE

Luxury eastern wear needs editorial storytelling, collection-led discovery, and careful product presentation alongside reliable shopping. Products, variants, cart state, checkout, and customer records must remain connected to commerce infrastructure, while campaigns need a flexible editorial layer.

THE SOLUTION

Built a Hydrogen storefront on Shopify Oxygen with React Router loaders and actions coordinating commerce, customer identity, and editorial content. Shopify owns products, transactions, and customer records; BILDIT supplies homepage hero and promotional slots with authored fallbacks. Critical route data is awaited while supporting sections stream through Suspense and Await, within an English/Pakistan market context.

KEY HIGHLIGHTS

- ✓ Editorial homepage combines campaign storytelling, featured collections, new arrivals, and best sellers.
- ✓ Shopify-backed collection filters, sorting, cursor pagination, and regular/predictive search.
- ✓ Shared cart drawer and page use Hydrogen cart identity, optimistic state, discounts, and Shopify checkout.
- ✓ Judge.me supplies integrated product review display and submission.
- ✓ BILDIT-managed hero/promo slots fall back to authored storefront content when unavailable.
- ✓ Variant-aware product URLs, option selection, and connected add-to-cart behavior.
- ✓ Customer Account OAuth protects profiles, addresses, orders, and customer-metafield wishlists.
- ✓ Streamed rendering, nonce-based CSP, server-side sessions, and distinct public/private caching boundaries.

TECHNOLOGY STACK

Application	Hydrogen 2026.4.5, React 18, React Router 7, strict TypeScript
Commerce	Shopify Storefront API, Hydrogen cart and product-option primitives
Accounts	Shopify Customer Account API, OAuth, customer metafields
Integrations	BILDIT Hydrogen adapter, Judge.me
Presentation	Tailwind CSS v4, local Inter and Playfair Display fonts
Runtime	Shopify Oxygen, Vite 8, GraphQL codegen, TypeScript, ESLint

VIDEO WALKTHROUGH

Watch project video

Goodman Group

A server-first corporate portfolio connecting one group identity to five distinct businesses through bespoke profiles, clear discovery, and validated enquiries.

INDUSTRY / SYSTEM	SCOPE	STATUS
Corporate Portfolio Website	**Group · Five Company Profiles · Enquiries**	**Implemented**

THE CHALLENGE

A group website needs a coherent parent identity while preserving businesses with different stories and information needs. Pharmaceutical manufacturing, medical-equipment trading, chemicals, medical billing, and a pharmaceutical business need tailored profiles, but disconnected pages make the portfolio harder to explore.

THE SOLUTION

Built a Next.js App Router website with a group landing page, registry-driven directory, and five bespoke company profiles. Readonly typed records connect shared identity to navigation, profile lookup, enquiry choices, metadata, and sitemap entries. CompanyGroupFrame reuses group framing rather than imposing one profile renderer. Focused client components handle themes, motion, navigation, and enquiry feedback.

KEY HIGHLIGHTS

- ✓ Dedicated profiles for Goodman Laboratories, Geron Pharma, Goodman Medical Equipment, Wal Green Chemicals, and Goodman Billing.
- ✓ Shared navigation preserves continuity while each business controls its composition, content, and styling.
- ✓ Lazy-loaded Lenis/GSAP/ScrollTrigger, effect cleanup, reduced motion, and pointer-aware glow.
- ✓ Resend delivery confirms success only after provider receipt; configuration and delivery errors remain visible.
- ✓ Group storytelling covers portfolio, leadership, history, and business presence with direct company discovery.
- ✓ Persistent themes use pre-hydration initialization, system preferences, and company-palette synchronization.
- ✓ Shared form validation, origin/content-type checks, 16 KiB body limit, honeypot, and per-IP Redis limits.
- ✓ Route metadata, canonicals, Organization JSON-LD, sitemap, robots, and explicit unknown-company not-found pages.

TECHNOLOGY STACK

Application	Next.js 16 App Router, React 19, strict TypeScript
Content	Readonly TypeScript registry, Next.js metadata, Organization JSON-LD
Presentation	Tailwind CSS v4, company-scoped CSS, Lucide icons
Motion	Lenis, GSAP / ScrollTrigger, browser storage
Enquiries	Node.js route handler, shared validation, Upstash Redis, Resend
Quality	Vitest, Testing Library, Playwright / Axe, Next.js standalone output

VIDEO WALKTHROUGH

Watch project video

Inventory & Finance Management System

A full-stack desktop operations platform connecting serialized inventory, customer receivables, vendor payables, and business reporting.

INDUSTRY / SYSTEM	SCOPE	STATUS
Equipment Inventory & Finance	**Desktop App · Stock · Sales · Procurement**	**Implemented**

THE CHALLENGE

Businesses managing batteries, rectifier modules, and related equipment need stock, customer commitments, vendor obligations, and financial records to agree. Disconnected spreadsheets and paper ledgers lose serial-level status, reservations, partial receipts, and payment context. High-impact changes also need an explainable history.

THE SOLUTION

Built an Electron/React desktop platform with protected Express APIs and a PostgreSQL/Prisma operating record. Unique serialized units retain commercial and physical states. Invoices reserve stock and post customer ledger entries; purchase orders, bills, receipts, and vendor payments share linked records. Serializable transactions, row locks, retries, and four-decimal monetary values protect financial updates, while reversals and soft deletes preserve history.

HOW IT WORKS

1. Manual entry or validated Excel intake links each serial number to its product hierarchy, vendor, purchase order, condition, and location.
2. Invoices check credit limits and arithmetic, reserve units, and update ledgers; payments reconcile balances and voids reverse effects.
3. Multiple vendor bills cannot exceed the PO total; partial receipts and payment records determine purchasing progress and delivery completion.

KEY HIGHLIGHTS

- ✓ Serial-level inventory tracks specifications, condition, repair state, physical location, handover, and movement history.
- ✓ Template/Samhan Excel intake includes validation, pre-import backup, rollback, and manual review.
- ✓ Customer/vendor payments include overpayment protection, reversible voids, statements, and aging.
- ✓ JWT, bcrypt, permissions, role-aware settings, user management, and audited business changes.
- ✓ Available, Reserved, Sold, Delivered, Under Repair, and Returned business states.
- ✓ Invoices support Draft, Sent, Partial, Paid, Overdue, and Cancelled states, tax, discounts, and due dates.
- ✓ Dashboard/report views cover sales, cash, customers, valuation, turnover, gross margin, and AR aging.
- ✓ Invoice, purchase-order, and vendor-bill PDFs; secure preload channels and managed localhost backend lifecycle.

TECHNOLOGY STACK

Desktop	Electron 27, Node child process, secure preload
Frontend	React 18, React Router, Ant Design, Zustand, React Query, Recharts
API	Node.js, Express, Helmet, CORS, rate limiting
Data / integrity	PostgreSQL, Prisma, Supabase or local Podman, DECIMAL(18,4), row locks
Exchange	ExcelJS / xlsx, Multer, PDFKit, Day.js

VIDEO WALKTHROUGH

Watch project video

Trulle Portal

A cinematic WebGL navigation system turning continuous line-art into five interactive peephole portals, packaged as a configurable WordPress/Elementor plugin.

INDUSTRY / SYSTEM	SCOPE	STATUS
Frontend Motion & Rendering	**WebGL · Animation · WordPress Plugin**	**Deployed**

THE CHALLENGE

Conventional menus interrupt a brand built around one continuous piece of line-art. Static hotspots feel detached from the illustration. The client needed the artwork itself to become navigation, with hidden previews and seamless destination transitions, delivered through Elementor so editors could change content without code.

THE SOLUTION

Built coordinated artwork, WebGL rendering, interaction, and editor-configuration layers. A full-bleed 4K SVG frames hidden media; PixiJS renders rings, masks, and preview sprites; invisible hit-boxes own interaction. Each widget gets a Zustand store through React context. Pixi's ticker is stopped so one GSAP ticker drives rendering and cached portal timelines without competing animation loops.

HOW IT WORKS

1. Hover advances a cached rest/hover/open timeline to reveal preview media. Ring growth redraws radius geometry each frame to keep stroke width crisp.
2. Click expands the mask beyond the viewport and cross-fades to one reusable destination iframe; resolved target keys de-duplicate preload/open requests.
3. Browser history and a short-lived session cache restore the open state before reverse animation, avoiding a flash back to rest.

KEY HIGHLIGHTS

- ✓ Peephole previews and click-to-destination expansion preserve the artwork-led interface.
- ✓ Image/video destinations use generated srcdoc documents; plain URLs load through iframe.src.
- ✓ Runtime-injected destination, preview media, label, text color, and enabled state.
- ✓ Field-level precedence merges JSON, attribute overrides, and Elementor-mapped configuration.
- ✓ Up to five configurable portals per widget, with multiple isolated instances on a page.
- ✓ Reusable paused GSAP timelines replace newly created animations on every interaction.
- ✓ Elementor repeater fields let non-developer editors configure portals without JSON edits.
- ✓ Explicit handling for repeated hovers, preload de-duplication, browser Back, and hard-reload recovery.

TECHNOLOGY STACK

Rendering	PixiJS 8 / WebGL, static 4K SVG artwork
Animation	GSAP, @gsap/react, one render ticker and cached timelines
UI / state	React 19, Zustand 5, per-instance React context
Build	Vite 6, self-contained IIFE library bundle and CSS
Integration	PHP WordPress plugin, Elementor repeater GUI, normalized data-portals JSON

VIDEO WALKTHROUGH

Watch project video

Trend Finder

A local-first personal research workspace combining Google Trends and Reddit signals to rank home-office product ideas and revisit saved scans.

INDUSTRY / SYSTEM	SCOPE	STATUS
Personal Product Research	**Trends · Reddit · Rankings · History**	**Implemented**

THE CHALLENGE

Search interest and community conversations offer different perspectives on a product idea. Switching between isolated searches makes candidates harder to compare consistently, research difficult to repeat, and earlier findings easy to lose. This hobby project brings keywords, demand signals, and scan history into one focused personal workspace.

THE SOLUTION

Built a React dashboard and local FastAPI service with configurable keywords, synonyms, communities, and lookback settings. Collectors compare recent Google Trends interest with earlier interest and count distinct matching Reddit posts and comments. Signals are min-max normalized within the current keyword batch and combined as 60% Trends and 40% Reddit on a 0-100 scale. SQLite retains raw signals, composite scores, and scan runs for later review.

HOW IT WORKS

1. Configure candidate keywords and synonyms, up to five subreddits, and a shared lookback window from one to twenty-four months.
2. POST /scan streams per-keyword Server-Sent Events as collectors move through pending, scanning, completed, or failed states.
3. GET /results refreshes the ranking; candidate details show component signals and saved composite-score history.

KEY HIGHLIGHTS

- ✓ Ranked home-office candidate list with the latest scan timestamp and score-based summaries.
- ✓ PRAW searches selected Reddit communities for keywords and synonyms in posts and comments.
- ✓ Live scan panel exposes per-keyword progress, collected values, failures, and overall completion.
- ✓ Saved scans make research repeatable; rankings are relative to each scan batch.
- ✓ Google Trends direction compares recent and earlier search interest within the selected window.
- ✓ Editable keywords, synonyms, communities, and lookback settings require no code changes.
- ✓ Detail views show raw Trends values, Reddit mentions, composite scores, and custom history charts.
- ✓ Loading, empty, error, and progress states support a single-user local research loop.

TECHNOLOGY STACK

Interface	React, TypeScript, Vite, React Router
Presentation	Tailwind CSS, inline styles, custom SVG history charts
API / workflow	Python, FastAPI, Pydantic, Server-Sent Events
Collectors	pytrends, PRAW, python-dotenv
Persistence	SQLAlchemy, SQLite: settings, keywords, communities, scan runs and results
Local execution	Uvicorn, Vite dev server, PowerShell launcher, automatic table creation and seeding

VIDEO WALKTHROUGH

Watch project video

Software Server Queue Simulation Dashboard

A local interactive simulator exploring how request traffic, service duration, and worker capacity affect waiting, response time, and SLA breaches.

INDUSTRY / SYSTEM	SCOPE	STATUS
Queueing Simulation / Learning Tool	**Workloads · Queues · Metrics · Comparisons**	**Implemented**

THE CHALLENGE

Queueing formulas do not make growing congestion easy to see. Small changes in arrival rate or worker count can create a large backlog. This hobby project turns those abstract relationships into reproducible experiments with request-level evidence, rather than live infrastructure monitoring or automatic scaling.

THE SOLUTION

Built a self-contained Streamlit dashboard around an M/M/c-style discrete-event model. NumPy generates seeded exponential interarrival intervals and service durations; SimPy processes requests through a bounded worker pool. The same workload is reused for baseline and recommended capacity. pandas calculates metrics and Plotly visualizes queue history, response-time distribution, and capacity comparisons.

HOW IT WORKS

1. A submitted sidebar form applies arrival rate, mean service time, workers, arrival duration, SLA threshold, and seed; presets offer quick starting points.
2. Arrivals stop at the configured duration, but every arrived request drains to completion. Queue averages use the full observed horizon; throughput uses the arrival duration.
3. A visible heuristic chooses the smallest worker count at or below 85% theoretical utilization, capped at ten. It does not guarantee the selected SLA.

KEY HIGHLIGHTS

- ✓ Healthy, Overloaded, and Optimized presets, editable controls, and session-held active parameters.
- ✓ Event-based step chart for queue history and a response histogram with an SLA threshold.
- ✓ First fifty completed-request records expose arrivals, starts, completions, waiting, and service durations.
- ✓ Simulated outcomes represent one workload, not real-server benchmarks; the source reports 51 passing tests.
- ✓ Seven KPIs: completed requests, average waiting/response, weighted queue length, utilization, throughput, and SLA breach rate.
- ✓ Identical demand isolates worker-count effects in the comparison table and grouped bar chart.
- ✓ Seed-42 example: 2 to 4 workers reduces mean waiting from 742.64s to 1.92s; the 5s SLA breach rate remains 51.84%.
- ✓ Local execution needs no external APIs, database, cloud service, authentication, or internet after setup.

TECHNOLOGY STACK

Application	Python, Streamlit, sidebar forms and session state
Workload	NumPy random generator, seeded exponential arrival/service samples
Simulation	SimPy Environment / Resource, discrete events and parallel workers
Analysis	pandas, Python math, explicit metric denominators and capacity heuristic
Visualization	Plotly graph objects / express
Verification	pytest, pytest-cov: workload, simulation, metrics, scaling, charts and smoke coverage

VIDEO WALKTHROUGH

Watch project video